图论

qwaszx

2026 年 9 月 18 日

目录

第一章 图论基础 **3**

1.1 基本概念 . . . 3
1.2 图的存储 . . . 3
1.3 图的遍历 . . . 3
1.3.1 DFS 树 . . . 3
1.3.2 BFS 树 . . . 3

第二章 连通性问题 **4**

2.1 无向图连通性 . . . 4
2.2 无向图双连通性 . . . 4
2.2.1 边双连通 . . . 4
2.2.2 点双连通 . . . 5
2.2.3 DFS 树与双连通性 . . . 5
2.2.4 耳分解 . . . 6
2.3 有向图连通性 . . . 7
2.4 有向无环图 . . . 9

第三章 生成树问题 **11**

3.1 基本概念和性质 . . . 11
3.2 算法 . . . 11
3.3 Kruskal 与连通性 . . . 12
3.4 生成树的边交换 . . . 13
3.5 稠密图的 MST . . . 14
3.6 拟阵 . . . 15

第四章 最短路问题 **16**

4.1 基本概念 . . . 16
4.2 全点对最短路 . . . 16
4.3 单源最短路 . . . 18
4.3.1 Dijkstra . . . 18
4.3.2 Bellman-Ford . . . 19
4.4 预处理和后处理技巧 . . . 20
4.4.1 超级源汇与反图 . . . 20
4.4.2 重赋权 . . . 20
4.4.3 最短路树 . . . 21
4.5 特殊图上的最短路算法 . . . 21
4.6 动态最短路 . . . 22

4.7 常用建模 22
4.7.1 差分约束 22
4.7.2 分层图 23
4.8 代数路径 23

第五章 特殊的图 **25**
5.1 树 25
5.2 二分图 25
5.3 DAG 25
5.4 欧拉图 25
5.5 竞赛图 25
5.6 平面图 25
5.7 广义串并联图 25

第六章 网络流 **26**
6.1 最大流问题 26
6.2 费用流 26

参考文献 **27**

1 图论基础

1.1 基本概念

1.2 图的存储

1.3 图的遍历

1.3.1 DFS 树

定理 1.1. 在无向图中，边可以根据 DFS 树分成两类：树边、回边（连接祖先和后代，亦称非树边）。

定义 1.2. 基本环. 每条非树边 (u,v) 与 (u,v) 在 DFS 树上的路径构成的环称为基本环。

评注. 实际上可以对任何一个生成森林定义。

定理 1.3. 如果一个子图的每个点度数都是偶数，则称这个子图是一个欧拉子图（注意没有要求连通）。任何欧拉子图都可以唯一地表为若干基本环的异或。

证明. 考虑任何欧拉子图 H，对其包含的每条非树边 e，将 H 异或上 e 对应的基本环。最终会得到一个只由树边构成的欧拉子图，树边构成的欧拉子图只能是空图。

由于每个基本环只能控制一条非树边，这种表示是唯一的。 □

练习 1.4. 给定一张无向图，计算其欧拉子图数量。

练习 1.5. P3907 环的异或.

定理 1.6. 在有向图中，边可以根据 DFS 树分成四类：树边、回边（从后代指向祖先）、前向边（从祖先指向后代）、横叉边（指向非祖先后代的先前结点）。

1.3.2 BFS 树

2 连通性问题

2.1 无向图连通性

需要将点按照所在的连通分量分类
静态：遍历每个连通分量
只加边：并查集
只删边（离线）：时间倒流，并查集
加边删边（离线）：线段树分治
加边删边（在线）：动态图连通性

2.2 无向图双连通性

2.2.1 边双连通

定义 2.1. 边双连通性. 对于无向图 G，若 u, v 在删除任意一条边后仍然连通，则称 u, v 是**边双连通**的。若 G 中任意两个点都是边双连通的，则称 G 是**边双连通图**。

练习 2.2. 证明边双连通性是一个等价关系。

定义 2.3. 桥. 对于无向图 G，若删掉边 e 后 G 的连通分量数量增加，则称 e 是一个**桥**。

练习 2.4. 证明一条边是桥当且仅当它不在任何简单环上。

引理 2.5. 边双连通的刻画. 以下四点等价：

- G 是边双连通图
- G 中没有桥
- G 中任何边都在一个简单环上
- G 中任意两点之间都有两条边不交的路径。

定义 2.6. 边双连通分量. 一个极大（在包含关系下）的边双连通子图称为一个**边双连通分量**。在 OI 中通常简称为“边双”。

练习 2.7. 边双缩点.

- 说明边双连通分量构成了点集的一个划分。
- 证明该划分与边连通性的等价类划分相同。
- 证明该划分与删掉所有桥后得到的连通块集合相同。
- 证明把每个边双连通分量缩成一个点后会得到一棵恰包含所有桥的树。

2.2.2　点双连通

练习 2.8. 在有至少三个点的图中，证明桥的至少一个端点是割点。

定义 2.9. 点双连通性. 对于无向图 G，若 u, v 在删除任意一个点后仍然连通，则称 u, v 是**点双连通**的。若 G 中任意两个点都是点双连通的，则称 G 是**点双连通图**。

评注. 注意点双连通性**不是**等价关系。

定义 2.10. 割点. 对于无向图 G，若删掉点 v 后 G 的连通分量数量增加，则称 v 是一个**割点**。

评注. 在 G 只有两个或一个点时，点双连通和割点定义会变得比较微妙。一般来说，我们认为只有一个点或两个点的连通图也是点双连通的。

练习 2.11. 在 $|V(G)| \geq 3$ 时，证明

- 桥 e 的端点 u 是割点当且仅当删除 e 后 u 所在的连通块大小至少为 2
- 点双连通蕴含边双连通

引理 2.12. 点双连通的刻画. 设 G 至少有三个点。以下四条等价：

- G 是点双连通图
- G 中没有割点
- G 中任意两点之间都有两条除起终点外点不交的路径。
- G 中任意两点都在一个简单环上

定义 2.13. 点双连通分量. 一个极大（在包含关系下）的点双连通子图称为一个**点双连通分量**。在 OI 中通常简称为“点双”。

练习 2.14. 点双缩点.

- 证明两个点双连通分量至多有一个交点，且该交点是割点
- 证明一条边或一个简单环恰好属于一个点双连通分量
- 对每个点双连通分量 S 建立一个方点，原图中的点 u 建立一个圆点。若 $u \in S$ 则在 u, S 之间连无向边。证明这样会得到一棵树。这就是**圆方树（Block-Cut Tree）**。

2.2.3　DFS 树与双连通性

引理 2.15. 一个双连通分量中最早被 DFS 到的点是这个双连通分量中所有点在 DFS 树中的祖先

证明. 设这个点为 u，考虑其所在双连通分量 B 中的另一个点 v。由于 B 连通，故存在一条完全位于 B 中的路径 $u \rightsquigarrow v$，从而 DFS 的性质说明 v 在 u 的子树中。 □

推论 2.16. 一个双连通分量在 DFS 树中是一个连通块。

证明. 考虑双连通分量中最早被 DFS 到的点 u 和另一个点 v，我们说明 u,v 之间的树路径 P 上的所有点和边都在双连通分量内即可。

边双连通的情况：若 P 中存在一个桥，则删除后 u,v 将处于不同的连通块，从而 u,v 不是边双连通的，矛盾。

点双连通的情况：设 u,v 在同一个点双连通分量 B 中，考虑 P 上任何不等于 u,v 的点 x。删掉 x 后 u,v 仍然连通，故存在一条 u 到 v 且完全位于 $B-\{x\}$ 中的简单路径 Q。取 Q 在 x 两侧与 P 相交的最近两个点 s,t，则 Q 上的 $s\rightsquigarrow t$ 段与 P 上的 $s\rightsquigarrow t$ 段拼起来就得到了一个包含 x 的简单环 C。由于 Q 完全位于 B 中且与 C 至少交于 s,t 两点，故 B 和 C 至少有两个交点，从而 $B\cup C$ 也是点双连通的，故 x 也在这个点双连通分量中。 □

我们可以在 DFS 的过程中维护目前的连通块划分。当加入新的边时，有可能导致一些根不再能够成为根，即发生了连通块的合并。对于双连通分量，合并的判据如下：

引理 2.17. 对于一条非树边，其对应的树路径上的所有点都在同一个边双连通分量中，所有边都在同一个点双连通分量中。

不难发现这样的合并都发生在整棵树根到当前点的路径上，且合并的都是一段后缀。我们维护根到当前点路径上每个块的根（对于边双，根是一个点；对于点双，边是一个根），需要合并时将根进行合并。根据 DFS 的性质，这些块中的点/边已经按照被 DFS 到的顺序划分好了；换句话说，如果我们把所有这些块中的点/边按照被 DFS 到的顺序排列，那么每个块都对应一个连续段，且块的根是连续段中的第一个。于是可以用一个栈来维护这些块中的点/边，只需要维护出所有根就能知道每个块中的点/边。

当一个点/边 DFS 结束后，如果它还是一个根，那么其对应的块以后不会再继续合并，将这个块弹出栈并输出。特别地，对于点双，需要注意根是边而不是点，即弹出这条边的浅点后还要再加回来。

以上描述的过程实际上更接近 Gabow 算法，其显式维护了根的合并。常用的 Tarjan 算法则是通过维护子树内由非树边可达的最小 `dfn` 来判断一个点/边是否需要合并到父亲上的。

2.2.4　耳分解

定义 2.18. 耳和耳分解. 图的一个**耳分解**是一个由一个圈和若干个简单路径或简单环组成的序列 $(C,P_1,\ldots,P_k)$，其中 C 是一个简单环，P_i 是一个起点和终点在 $C\cup\bigcup_{j=1}^{i-1}P_j$ 中、中间点都不在 $C\cup\bigcup_{j=1}^{i-1}P_j$ 中的简单路径或简单环（环的起点和终点视为同一个点）。这样的 P_i 称为**耳（Ear）**。

定理 2.19. 一个图是边双连通的当且仅当存在耳分解。

证明. 存在耳分解时，显然每条边都在一个环中。反过来，假设图是边双连通的。找一个极大的存在耳分解的子图 H 并假设 $H\neq G$。注意若 $e\notin H$ 但是 e 的两个端点都在 H 中，则这条边本身就是一个耳，故 H 是 $V(H)$ 的导出子图。任取一个 $(H,G\backslash H)$ 之间的边 $e=(u,v)$，则存在一条 v 到 u 的不经过 e 的路径 P，取 P 进入 H 之前的一段 P_0，则 $e+P_0$ 是一个耳，与 H 极大矛盾。 □

对于点双连通图，有类似的结果成立。

定义 2.20. 开耳和开耳分解. 图的一个**开耳分解**是一个由一个圈和若干个简单路径组成的序列 $(C, P_1, \ldots, P_k)$，其中 C 是一个简单环，P_i 是一个起点和终点在 $C \cup \bigcup_{j=1}^{i-1} P_j$ 中、中间点都不在 $C \cup \bigcup_{j=1}^{i-1} P_j$ 中的简单路径。这样的 P_i 称为**开耳（Open Ear）**。

评注. 也就是说不再允许 P_i 是环。

定理 2.21. 对于有至少三个点的无向图 G，G 是点双连通的当且仅当存在开耳分解。

证明. 存在开耳分解时，逐个开耳加进去，可以始终维持点双连通。

反过来，假设图是点双连通的。找一个极大的存在开耳分解的子图 H 并假设 $H \neq G$。注意若 $e \notin H$ 但是 e 的两个端点都在 H 中，则这条边本身就是一个开耳，故 H 是 $V(H)$ 的导出子图。任取一个 $(H, G \backslash H)$ 之间的边 $e = (u, v)$，则存在一条 v 到 u 的不经过 e 且经过 H 中 v 以外的某个点的路径 P，取 P 进入 H 之前的一段 P_0，则 $e + P_0$ 是一个开耳，与 H 极大矛盾。 □

以下算法可以计算耳分解。

算法 2.22. 耳分解. 任取一棵 DFS 生成树，初始将所有点标记为未访问。按照 DFS 序遍历所有点 u，然后遍历所有与 u 相连的非树边 (u, v)。从 v 到 u 遍历树路径上的点并标记为已访问，直到遇到一个已访问的点，这些访问到的点构成一个耳。

定理 2.23.

- G 边（点）双连通当且仅当此算法给出一个（开）耳分解
- 一条边是桥当且仅当它不在任何算法得到的耳中
- 若 G 的所有点度数都 ≥ 2，则一个点是割点当且仅当它是一条桥边的端点或者是算法得到的某个耳的起点。

耳分解提供了理解边双和点双的十分方便的视角。

例 2.24. 强连通定向. 给一张无向图的边定向，使其成为强连通图。证明可以强连通定向 $\Longrightarrow$ 边双连通 $\Longrightarrow$ 存在耳分解 $\Longrightarrow$ 可以强连通定向。

2.3　有向图连通性

定义 2.25. 连通性.
在有向图中，我们定义以下几种关系：

- 弱连通性：若 u, v 在忽略边的方向得到的无向图中连通，则称 u, v 是弱连通的。
- 可达性：如果存在从 u 到 v 的有向路径，则称 u 可以到达 v。
- 强连通性：如果 u, v 可以互相到达，则称 u, v 是强连通的。

练习 2.26. 证明弱连通和强连通都是等价关系。

定义 2.27. 强连通分量. 由强连通性划分出的等价类称为强连通分量（Strongly Connected Component, SCC）。

定理 2.28. 把每个 SCC 缩成一个点后会得到一个有向无环图（DAG）。

与无向图的双连通性类似，我们可以利用 DFS 树的性质求出强连通分量。

性质 2.29. 在树边、回边、前向边、横叉边的分类下，只有树边和前向边可以到达 DFS 序比自己大的点，且到达的必定是子树内的点。

引理 2.30. 一个 SCC 中最早被 DFS 到的点是这个 SCC 中所有点在 DFS 树中的祖先

证明. 设这个点为 u，考虑 SCC 中的另一个点 v。对于任何从 u 到 v 的路径，这条路径上不可能包含比 u 更早被 DFS 到的点 w，否则可以得到回路 $u \to w \to v \to u$，从而 w 也在 SCC 中，与 u 是最早 DFS 到的点矛盾。于是在 `dfs`(u) 结束后，所有这些点 v 都会被通过某条路径 $u \to v$ 发现，从而 v 在 u 的子树中。

□

推论 2.31. 一个 SCC 在 DFS 树中是一个连通块。

证明. 考虑 SCC 中最早被 DFS 到的点 u，则 u 是 SCC 中所有点的祖先。若 v 在这个 SCC 中，则对于任何 u, v 路径上的点 w，都存在 $u \to w \to v \to u$ 的路径，故 w 也在这个 SCC 中。 □

引理 2.32. 按照结束 DFS 的时间从大到小排序可以得到缩点后 DAG 的一个拓扑序。

算法 2.33. Kosaraju 算法. 首先用一遍 DFS 算出此拓扑序，然后按照拓扑序在反图上 DFS，每次新访问到的点构成一个 SCC。

算法 2.34. Tarjan 算法. 在 DFS 时维护目前还没有“封闭”为 SCC 的候选点集栈。一个点可以成为一个 SCC 的根的充要条件是它无法沿着一条只经过子树与候选点集的路径到达它的祖先。找到一个根时，其子树对应候选点集栈的末尾一段，将其构建为一个 SCC 并从栈中弹出即可。
维护一个数组 low 表示每个点沿着只经过子树与候选点集的路径可以到达的最浅祖先即可。通常用最小 DFS 时间戳 dfn 来记录最浅祖先。

代码 2.1: Tarjan 算法

```
1 void tarjan(int u){
2     dfn[u]=low[u]=++time_stamp;
3     stk.push(u);
4     in_stk[u]=true;
```

```
    for(int v:G[u]){
        if(!dfn[v]){
            tarjan(v);
            low[u]=min(low[u],low[v]);
        }else if(in_stk[v]){
            low[u]=min(low[u],low[v]);
        }
    }
    if(dfn[u]==low[u]){
        // u 是一个 SCC 的根
        while(stk.top()!=u){
            int v=stk.top();stk.pop();
            in_stk[v]=false;
            // v 属于这个 SCC
        }
        stk.pop();
        in_stk[u]=false;
        // u 属于这个 SCC
    }
}
```

评注. *在标准的 Tarjan 算法中，在 `in_stk[v]` 时使用 `dfn[v]` 来更新 `low[u]`，而按我们的描述则是用 `low[v]` 更新 `low[u]`。证明它们都是正确的。*

算法 2.35. Gabow 算法. 在 Tarjan 算法中，直接使用一个单调栈来维护目前候选点集的每个候选 SCC 的根。如果 u 能够到达一个目前候选点集中的点 v，则所有 $\mathrm{dfn} > \mathrm{dfn}_v$ 的点都无法成为 SCC 的根，将其从单调栈中弹出即可。

2.4　有向无环图

DAG 给出了一个偏序关系。可以通过**拓扑排序**将其扩展为一个全序。任意一个这样的全序都称为一个**拓扑序**。通过按照拓扑序 DP，以下问题都可以快速计算：

- 最短路
- 最长路
- 从每个点出发的路径数量

很多一般有向图上的问题可以通过缩点等方式变为 DAG 上的问题。

例 2.36. DAG 可达性/传递闭包. 对每个点计算出它能到达的结点集合。$O(nm/w)$。

练习 2.37. 对每个点计算它能到达的结点数量。需要线性空间。

例 2.38. QOJ4420. Range Reachability Query. 有一个 DAG。每次询问给出 x_i, y_i, l_i, r_i，判断能否从 x_i 只经过编号在 $[l_i, r_i]$ 内的边到达 y_i。

解答. 设 $f_{x,i}$ 表示能否从 x 出发只经过编号在 $[l_i, r_i]$ 内的边到达 y_i。则对 $e_j = (u, v)$ 转移时的方程为 $f_{u,i}| = f_{v,i}\&[j \in [l_i, r_i]]$。使用 bitset 优化，需要对每个 j 计算出 $p_{j,i} = [j \in [l_i, r_i]]$ 对应的 bitset。这里可以对 $p'_{j,i} = [j = l_i] + [j = r_i + 1]$ 的第一维做前缀异或和得到。

最后可以对询问按 $B \geq w$ 大小分块做到 $O(\frac{mq}{w})$ 时间和 $O(mB/w)$ 空间。

3 生成树问题

3.1 基本概念和性质

定义 3.1. 生成树. 对于无向图 G，如果它的一个生成子图 H 是树，则称 H 是一棵生成树（spanning tree）。
如果一个子图 H 是由 G 的每个极大连通块的一个生成树组成的，则称 H 是原图的一个生成森林。

评注. 一般来说，考察生成树就能自动得到生成森林的对应结论。因此我们通常只需考虑生成树。在本章中，我们都假设图连通。

性质 3.2. 割与生成树. 对于图 G 中的一个非空点集 $S \subset V$，S 和 $V\backslash S$ 之间的边称为 G 的一个**割** (cut)，记作 $\mathrm{cut}(S, V\backslash S)$。一棵生成树必定包含 $\mathrm{cut}(S, V\backslash S)$ 中的至少一条边。

DFS 树就是一棵生成树。因此，如果只需要找出任意一棵生成树，可以使用 DFS 树。

定义 3.3. 最小生成树. 对于带权无向图 G 的所有生成树，边权和最小的那些被称为最小生成树（minimum spanning tree，MST）。

定理 3.4. 割性质. 对任何 S，考虑 $\mathrm{cut}(S, V\backslash S)$ 中权值最小的某条边 e，它必定被某个 MST 包含；反过来，每个 MST 都包含 $\mathrm{cut}(S, V\backslash S)$ 中权值最小的边中的至少一条。特别地，如果这样的边只有一条，则所有 MST 都包含它。

证明. 考虑任何一个 MST $\mathcal{T}$，和 $\mathrm{cut}(S, V\backslash S)$ 中权值最小的某条边 e。如果 $e \in \mathcal{T}$，则结论成立。否则，设 $e = (u, v)$，考虑 $\mathcal{T}$ 中 u 到 v 的路径，其中必然有一条边 $e' \in \mathrm{cut}(S, V\backslash S)$。将 e' 替换为 e 仍然得到一棵生成树，且不会变劣。可以类似地说明每个 MST 都包含 $\mathrm{cut}(S, V\backslash S)$ 中权值最小的边中的至少一条。 □

3.2 算法

使用割性质可以得到几种不同的求 MST 的算法。

算法 3.5. Prim. 维护点集 S，初始令 $S = \{1\}$。每次寻找 $\mathrm{cut}(S, V\backslash S)$ 中权值最小的边，将其加入生成树，对应的点加入 S。如此反复直到 $S = V$。
$O(n^2)$（暴力）或 $O(m \log m)$（优先队列）或 $O(m + n \log n)$（支持 $O(1)$ decrease-key 的堆）。

算法 3.6. Borůvka. 维护一个连通块划分 $\{S_1, S_2, \ldots, S_k\}$，初始令每个点为一个连通块。每次对每个连通块 S_i，选择 $\mathrm{cut}(S_i, V\backslash S_i)$ 中权值最小的边加入生成树。如此反复直到 $k = 1$。

至多 $\log n$ 轮，直接实现即为 $O(m \log n)$

算法 3.7. Kruskal. 将所有边按权值从小到大排序，依次考虑每条边，如果它连接的两个点不在同一个连通块中，则将其加入生成树。
$O(m(\log m + \alpha(m,n)))$（排序、并查集）

证明. 一种理解 Kruskal 正确性的方式是，将其看做“异步”的 Borůvka 算法，也就是每次在所有 $\mathrm{cut}(S_i, V \backslash S_i)$ 中取一个权值最小的边。由于这样取出来的边权必然是不降的，我们可以改为从小到大扫描所有边并判断每条边是否合法。

除了这种理解以外，我们也给出另一个证明，它体现了生成树作为**拟阵**而具有的性质。设 Kruskal 算法过程中按顺序选择了 $e_1, \cdots, e_{n-1}$ 加入了生成树。我们逐步构造出一棵包含 $e_1, \cdots, e_i$ 的 MST $\mathcal{T}_i$。假设已经构造出 $\mathcal{T}_{i-1}$，考虑其边集 E_{i-1}。如果 $e_i \in E_{i-1}$，那么直接令 $\mathcal{T}_i = \mathcal{T}_{i-1}$ 即可。否则，设 $e_i = (u, v)$，考虑 u, v 在 $\mathcal{T}_{i-1}$ 中对应的路径。这条路径中必然有一条边 e 满足加入到 $\{e_1, \ldots, e_{i-1}\}$ 中后不会成环，否则它们无法令 (u, v) 连通。而根据 Kruskal 算法的选择顺序，e_i 的权值不大于 e 的权值。于是在 $\mathcal{T}_{i-1}$ 中将 e 替换为 e_i 得到的生成树 $\mathcal{T}_i$ 仍然是 MST。 □

评注. *在考虑 MST 的性质时，权值相同的边有时会造成麻烦。我们可以给这些边添加一个不同的第二关键字来进行比较，第二关键字的排序可以任意选择。对于整数边权的情况，还有一个做法是令 $\tilde{w}_{e_i} = w_{e_i} \cdot |E| + i$ 来让边权互不相同。*

3.3 Kruskal 与连通性

推论 3.8. 对于图 G，记 $G_{\le W}$ 为只保留权值 $\le W$ 的边得到的子图。则对于任何 G 的 MST $\mathcal{T}$，$\mathcal{T}_{\le W}$ 的连通情况都与 $G_{\le W}$ 相同。

证明. 按 W 归纳，假设 $< W$ 时成立，形成的连通块集合为 C。考虑所有权值等于 W 的边。Kruskal 算法构造的 MST $\mathcal{T}^k$ 中，$\mathcal{T}^k_{\le W}$ 的连通情况与 $G_{\le W}$ 相同。考虑任意一棵 MST $\mathcal{T}$，假设其不满足此结论。那么存在 C 中的两个连通块 α, β，其在 $\mathcal{T}^k_{\le W}$ 中连通，而在 $\mathcal{T}_{\le W}$ 中不连通。根据前面 Kruskal 算法的证明过程，可以将 $\mathcal{T}$ 中的某条权值大于 W 的边替换为 $\mathcal{T}^k_{\le W}$ 中连接 α, β 的边，这样得到一个权值严格更小的生成树，矛盾。 □

推论 3.9. 任何 MST 都可以通过给同权边按照某种方式定序后由 Kruskal 算法得到。所有 MST 的边权多重集都相同。

练习 3.10. 计算每条边是否能出现在 MST 中；判断 MST 是否唯一。

练习 3.11. P4208 [JSOI2008] 最小生成树计数. 假设你有一个 $O(n^3)$ 计算生成树数量的算法（即矩阵树定理），设计一个 $O(n^3)$ 计算最小生成树数量的算法。

定义 3.12. 最小瓶颈路与最小瓶颈生成树. 使得最大边权最小的生成树称为最小瓶颈生成树或瓶颈生成树。
在 u, v 之间的所有路径中，使得最大边权最小的路径称为 u, v 的最小瓶颈路。

评注. 注意这两个都可能不唯一。

定理 3.13. MST 是最小瓶颈生成树，反之则不然。MST 中的任意两点之间的路径都是最小瓶颈路，反之则不然。

可以使用 **Kruskal 重构树**来刻画每个 W 对应的 $G_{\leq W}$ 的连通性质。

数据结构 3.14. Kruskal 重构树. 从小到大加边合并连通块，过程中每个产生的连通块（包括初始的每个点）都用一个结点表示。一条边合并两个连通块时，将这两个连通块对应的结点作为新连通块对应结点的两个孩子，新结点的权值为这条边的权值。这样得到的二叉树称为 Kruskal 重构树，简称重构树。

性质 3.15.

- Kruskal 重构树是二叉树，叶子结点对应原图的点，非叶子结点对应某个连通块。
- 考虑最小的使得 u, v 在 $G_{\leq W}$ 中连通的 W，则 W 即为 u, v 在 Kruskal 重构树 LCA 的权值。
- 若边权互不相同，则 Kruskal 重构树中一个权值为 W 的结点对应的子树是 $G_{\leq W}$ 中的一个连通块。

例 3.16. P4768 [NOI2018] 归程.

练习 3.17. P1967 [NOIP 2013 提高组] 货车运输.

3.4　生成树的边交换

例 3.18. 对每条边求出包含/不包含它的生成树中边权和最小的一个。

例 3.19. (严格) 次小生成树. 计算权值和 (严格) 次小的一棵生成树。

引理 3.20. 任取一棵 MST $\mathcal{T}$，存在一棵 (严格) 次小生成树与 $\mathcal{T}$ 只差一条边。
该结论的证明参见section 3.6。

解答. 任取一个 MST，枚举一条非树边，从对应的树链中去掉一条边即可。

例 3.21. 增量 MST. 加边，查询 MST。

评注. 可以配合线段树分治支持删边。

3.5　稠密图的 MST

在一些问题中，我们需要对边权由特殊方式给定的规模较大的图计算 MST。此时，以下几个定理可以帮助缩减图的规模。

定理 3.22.

1. 割性质：如果 $H \subseteq G$ 包含了 G 的所有割的最小边，则 H 的 MST 也是 G 的 MST。
2. 环性质：一个环上严格最大的边必定不在任何 MST 中。特别地，如果 $\max(w(u,x), w(x,v)) < w(u,v)$，则 $w(u,v)$ 无用。
3. 合并性质：$\mathrm{MSF}(\mathrm{MSF}(G) \cup \mathrm{MSF}(H))$ 是 $G \cup H$ 的一个 MSF。

证明. 只证明合并性质。不妨设边权互不相同。显然左边 $\leq$ 右边，只需证明右边 $\leq$ 左边。考虑 G 中的一条权值为 W 的边 $e = (u,v)$，若 e 不在 G 的 MSF 中，则 $G_{<W}$ 中 u, v 已经连通，故 $(G \cup H)_{<W}$ 中 u, v 也已经连通，从而 e 也不在 $G \cup H$ 的 MSF 中。因此，只有 G 或者 H 的 MSF 中的边才有可能出现在 $G \cup H$ 的 MSF 中。 □

Borůvka 算法常常可以配合数据结构在 $\tilde{O}(|S|)$ 时间内计算出 $\mathrm{cut}(S, V \backslash S)$ 的最小边。因此，Borůvka 算法在处理这种问题时常常会更加方便。

例 3.23. 一张 n 个点的完全图，对以下每种方式定义的边权，计算 MST 的边权和。

- $w(i,j) = a_i + a_j$
- $w(i,j) = |a_i - a_j|$
- $w(i,j) = \max(a_i, a_j)$
- $w(i,j) = \min(a_i, a_j)$
- CF888G $w(i,j) = a_i \oplus a_j$
- $w(i,j) = -(a_i \oplus a_j)$
- P2076 $w(i,j) = |x_i - x_j| + |y_i - y_j|$
- $w(i,j) = -\gcd(a_i, a_j)$
- $^*w(i,j) = \gcd(a_i, a_j)$
- $w(i,j) = \mathrm{lcm}(i+1, j+1)$
- T182745 $w(i,j) = \max(a_i, a_j) \bmod \min(a_i, a_j)$

解答.

练习 3.24. $w(i,j) = \omega(\mathrm{lcm}(i,j))$，每次询问 $[l,r]$ 这些点的导出子图的 MST。

解答. 设 $S(i)$ 为 i 的质因子集合，则 $w(i,j) = |S(i) \cup S(j)|$。注意到 $w(i,j) \geq |S(i)|$，而对于素数 p 总有 $w(i,p) \leq |S(i)| + 1$。因此，如果 $[l,r]$ 中有素数，那么 $w(i,j) \geq |S(i)| + 1$ 的边都可以被 (i,p) 替代。所以我们对每个 i 就只需要考虑那些 $w(i,j) = |S(i)|$ 的边，这等价于 $S(j) \subseteq S(i)$。

下面假设 $[l,r]$ 中存在素数。首先将 $[l,r]$ 中的所有点按照 $S(i)$ 分类，每类内部可以先连起来。然后对于每个类 $|S|$，考虑它与其他类之间的连边。如果存在一个类 $T \subset S$，那么这个

割的最小边为 $|S|$，否则为 $|S|+1$。不难发现第一种连边构成的是一个有根森林，故可以直接把这些边连上，最后再把每个根连到素数上即可。

现在考察答案的形式，若有 m 个根，则不难发现答案为 $m-2+\sum_{i=l}^{r}\omega(i)$。设 $l_i=\max(j<i:S(j)\subset S(i))$，$r_i=\min(j>i:S(j)\subset S(i))$，则一个类可以成为根当且仅当这个类中的每个元素 i 都满足 $l_i<l$ 且 $r_i>r$。再记 $\mathrm{pre}_i=\max_{j<i:S(j)=S(i)}$，则进一步有

$$m=\sum_{i=l}^{r}[l_i<l\wedge r_i>r\wedge \mathrm{pre}_i<l],$$

这是一个三维数点问题，也可以通过容斥变成二维。l_i 和 r_i 的计算可以通过暴力枚举 $S(i)$ 的子集来寻找，这部分复杂度不超过 $\sum_i d(i)=O(n\log n)$。

最后，如果没有素数，那么在 OI 常用范围中 $r-l+1$ 都不大，暴力计算 MST 即可。

评注. n 以内的最大素数间距是多少？这个问题还是开放的。已经证明的范围是

$$\left[\Omega\left(\frac{\log n\log\log n\log^{(4)}n}{\log^{(3)}n}\right),O(n^{0.525})\right].$$

Cramér 猜想认为它是 $\Theta(\log^2 n)$ 的。

例 3.25. 离线动态 MST. P3206 [HNOI2010] 城市建设
支持修改一条边的权值，询问 MST。

3.6　拟阵

4 最短路问题

4.1 基本概念

定义 4.1. 最短路. 在所有从 u 到 v 的路径（未必简单）中，权值和最小的路径称为 u, v 的最短路，其权值和记作 $d(u, v)$。特别地，如果不存在 u 到 v 的路径，则令 $d(u, v) = +\infty$。需要注意可能存在 $d(u, v) = -\infty$ 的情况。

我们规定以下加法规则：对任何实数 x，

- $-\infty + (-\infty) = -\infty$
- $-\infty + x = -\infty$
- $-\infty + \infty = \infty$
- $\infty + x = \infty$
- $\infty + \infty = \infty$

有向图和无向图的最短路在许多情况下性质都是相同的。如无特殊说明，本章中提到的图既可以是有向图，也可以是无向图。

引理 4.2. $|d(u, v)| < \infty$ 当且仅当存在一条 u 到 v 的简单路径的权值和等于 $d(u, v)$，也即存在一条最短路为简单路径。

评注. 反过来说，$|d(u, v)| = \infty$ 当且仅当 u 不可达 v 或者存在一条从 u 到 v 的路径经过了一个负环。因此，在无负环的图上，只要 u, v 之间的路径存在，最短路就总可以是简单路径，于是此时 $d(u, v)$ 有限。

评注. 在有负环的图上，需要区分“最短路”和“最短简单路径”。前者可能经过一个点无数次，而后者只能经过一个点至多一次。注意计算最短简单路径在有负权的图上不弱于判定 *Hamilton* 路径的存在性，而这是一个 *NPC* 问题。这可以解释为什么我们会听到“最长路是 *NP* 难的”和“最长路就是边权取负后的最短路”这两种看似互相矛盾的说法，因为前者指的是最长简单路径，而后者指的是最长路。

性质 4.3. 三角不等式. 最短路长度满足 $d(u, v) \le d(u, w) + d(w, v)$。当 $d(u, v) < +\infty$ 时，等号成立当且仅当 w 在某条 $u \to v$ 的最短路上。

在有负环的情况下，许多问题将会失去意义。在本章中，除非特殊说明，我们默认图无负环。

4.2 全点对最短路

计算全点对最短路（All-Pairs Shortest Path, APSP）的一个简单有效的算法是 Floyd 算法。

算法 4.4. Floyd 算法. 令 $d_i(u,v)$ 为从 u 到 v 且经过的中间点编号都不超过 i 的点的最短路长度。则有转移方程：

$$d_i(u,v)=\min(d_{i-1}(u,v),d_{i-1}(u,i)+d_{i-1}(i,v))$$

其中 $d_0(u,v)$ 为初始的边权，即 $d_0(u,v)=w(u,v)$。最终答案为 $d_n(u,v)$。可以滚动数组省去 i 一维。时间 $O(n^3)$，空间 $O(n^2)$。

代码 4.1: Floyd 算法

```
// 没有边则 w=+inf
d=w;
for(int k=1;k<=n;k++)
    for(int i=1;i<=n;i++)
        for(int j=1;j<=n;j++)
            d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
```

评注. 这是要求最短路至少包含一条边时的情况。如果可以不包含边，那么还需要补充初值 $d_0(u,u)=0$。

评注. 目前，人们既不知道 *APSP* 是否有 $O(n^{3-\epsilon})$ 的算法，也不能证明 $\tilde{\Omega}(n^3)$ 的下界。因此，*APSP* 是精细复杂性（*Fine-Grained Complexity*）中的一个重要问题。有许多问题可以和 *APSP* 互相归约：

- 判断图中是否存在负权三角形
- *min+* 矩阵乘法
- 求图中最小环

练习 4.5. 在无负环的图上证明 Floyd 算法的正确性。进一步修改 Floyd 算法使其能够在有负环的图上正确计算 $d_i(u,v)$。

解答. 在计算过程中的任何时刻，若更新后产生了 $d[u][u]<0$，则令 $d[u][u]=-\infty$。

练习 4.6. 计算图中权值和最小的简单环，或者报告存在负环。

评注. 在存在负环的情况下，计算权值和最小的简单环不弱于判定 *Hamilton* 回路的存在性，这也是一个 *NPC* 问题。

解答. 对每个环，考虑其上编号最大的点 x，则 $d_{x-1}(u,x)+d_{x-1}(x,v)+d_{x-1}(v,u)$ 对应的环的权值不超过此环的权值。

练习 4.7. 对每个点 x，计算出删去 x 后的两两最短路。$O(n^3\log n)$。

4.3　单源最短路

计算单源最短路（Single-Source Shortest Path, SSSP）是一个更常见的问题，我们希望计算出从一个源点 s 到所有点的最短路长度 $d_s(v) = d(s,v)$。上下文清楚时，我们也直接写作 $d(v)$。

引理 4.8. Bellman 方程与松弛. 若 $\tilde{d} \geq d_s$，则 $\tilde{d} = d_s$ 当且仅当 $\tilde{d}(s) = 0$，且每个点 $v \neq s$ 都满足 Bellman 方程：$\tilde{d}(v) = \min_u \tilde{d}(u) + w(u,v)$。
$\tilde{d}(v) \leftarrow \min(\tilde{d}(v), \tilde{d}(u) + w(u,v))$ 称为对 v 的**松弛**操作。若松弛操作后 $\tilde{d}(v)$ 发生了变化，则称 v 可以被边 (u,v) 松弛。若 $\tilde{d} \geq d$ 且 $\tilde{d}(s) = 0$，则 $\tilde{d} = d$ 当且仅当 $\tilde{d}$ 无法被任何一条边松弛。

一大类最短路算法都可以理解为从 $\tilde{d}(u) = \begin{cases} 0 & u = s \\ +\infty & u \neq s \end{cases}$ 开始，按照某种顺序进行松弛操作来逼近真实最短路长度 $d(u)$。

4.3.1　Dijkstra

在无负权的图上，Dijkstra 算法可以在 $O(m + n\log n)$ 的时间内计算出单源最短路，常规的算法竞赛实现则是 $O((n+m)\log m)$ 的。其想法是每次选择未确定最短路的点中 $\tilde{d}(u)$ 最小的点 u，将 u 的最短路 $d(u)$ 确定为 $\tilde{d}(u)$，然后 u 的所有出边进行松弛。

Dijkstra 的发音是 /ˈdaɪkstrə/

引理 4.9. 在无负权图上，选中 u 时有 $\tilde{d}(u) = d(u)$。

证明. 设 S 为已确定最短路的点集，$\tilde{d}$ 可以视为维护了从 s 出发且中间只经过 S 中的点时的最短路。对于选中的 u，考虑任何从 s 到 u 的路径。设该路径上第一个不在 S 中的点为 v，则这部分路径长度至少是 $\tilde{d}(v) \geq \tilde{d}(u)$，且 $v \to u$ 这一段的总长度非负。于是 $\tilde{d}(u)$ 即为最短路的长度。 □

算法 4.10. Dijkstra 算法. 令 S 为最短路已经确定的点集，初始 $S = \varnothing$。初始化 $\tilde{d}(s) = 0$，$\tilde{d}(v) = +\infty$（$v \neq s$）。
每次从 $V \backslash S$ 中选择 $\tilde{d}(u)$ 最小的点 u 加入 S，然后枚举所有出边 (u,v) 进行松弛。
使用数据结构维护 $V \backslash S$ 的 $\tilde{d}$。暴力做是 $O(n^2)$，用优先队列是 $O((n+m)\log m)$，用支持 $O(1)$ 插入和 decrease-key、$O(\log n)$ 删除的堆（如斐波那契堆）是 $O(m + n\log n)$。

代码 4.2: Dijkstra 算法

```
for(int i=1;i<=n;i++) d[i]=inf;
d[s]=0;
// 优先队列
priority_queue<pair<int,int>,vector<pair<int,int> >,greater<pair<int,int>
    > > pq;
pq.push(make_pair(0,s));
```

```
while(!pq.empty()){
    int u=pq.top().second;
    pq.pop();
    if(vis[u]) continue;
    vis[u]=1;
    for(auto [v,w]:e[u]){
        if(d[v]>d[u]+w){
            // decrease-key, 在优先队列中只能实现成插入
            d[v]=d[u]+w;
            pq.push(make_pair(d[v],v));
        }
    }
}
```

评注. 由于大多数情况下图无负权，*Dijkstra* 算法是实践中最常用的最短路算法。

Dijkstra 算法直到 *2025* 年为止都是计算无负权有向图上单源最短路的复杂度最好的算法。目前，无负权图上的 *SSSP* 在理论上可以做到 $O(m\sqrt{\log n}+\sqrt{mn\log n\log\log n})$，参见此论文。

练习 4.11. 在 $O(nk^2+m\log n)$ 复杂度内计算离每个点前 k 近的点。

4.3.2 Bellman-Ford

在有负权的图上，可以使用 Bellman-Ford 算法计算单源最短路，其想法在于递推计算经过至多 i 条边的最短路。每轮用每条边松弛一次即可得到至多经过 $i+1$ 条边的最短路。若图中无负环，则 $n-1$ 轮后可以正确计算出最短路。反过来，若第 n 轮仍能成功松弛，则说明存在负环。

算法 4.12. Bellman-Ford 算法. 初始化 $\tilde{d}(s)=0$，$\tilde{d}(v)=+\infty$（$v\neq s$）。重复 $n-1$ 轮，每轮用每条边 (u,v) 进行松弛。若第 n 轮仍能成功松弛，则说明存在负环。
时间复杂度 $O(nm)$，空间 $O(n+m)$。

评注. 第 n 轮仍能成功松弛当且仅当 s 能到达一个负环。因此，如果需要判断整个图中是否存在负环，需要添加一个向每个点连 0 边的超级源点 ss，然后从 ss 出发运行 *Bellman-Ford* 算法。

评注. *Bellman-Ford* 算法直到 *2022* 年为止都是计算有负权图上单源最短路最快速的算法。目前，有负权图上的 *SSSP* 在理论上可以做到 $O(m\log^2 n\log(nW)\log\log n)$（参见此论文）或 $m^{1+o(1)}$（参见此论文）。

练习 4.13. 计算从 s 出发经过至多/恰好 k 条边的最短路。$O(km)$。

由于只有被成功松弛的结点才有可能进一步对其出边进行松弛，我们可以调整松弛结点的顺序。一种方式是像 BFS 一样进行松弛，这就是队列优化的 Bellman-Ford 算法。在国内

也常被称为 SPFA 算法。

算法 4.14. 队列优化的 Bellman-Ford 算法/SPFA. 初始化 $\tilde{d}(s)=0$，$\tilde{d}(v)=+\infty$（$v\neq s$）。维护一个队列 q，初始将 s 入队。每次从队列中取出一个点 u，用其所有出边 (u,v) 进行松弛，若成功松弛且 v 不在队列中，则将 v 入队。若某个点入队次数达到了 n 或者最短路边数达到了 n，则说明存在负环。
最坏时间复杂度为 $O(nm)$，空间 $O(n+m)$。

评注. *虽然 SPFA 的最坏时间复杂度与 Bellman-Ford 相同，但对于没有认真构造的数据，SPFA 常常有良好的表现。*

4.4　预处理和后处理技巧

4.4.1　超级源汇与反图

例 4.15. 给一个点集 S，对每个点 u 计算 $\min_{x\in S} w_x+d(x,u)$。

例 4.16. 给一个点集 S，对每个点 u 计算 $\min_{x\in S} d(u,x)+w_x$。

练习 4.17. P5304 [GXOI/GZOI2019] 旅行者. 正权图，给一个点集 S，计算 $\min_{x,y\in S,x\neq y} d(x,y)$

解答. 令 $U_{i,b}=\{x\in S:x_i=b\}$，其中 x_i 代表 x 在二进制下的第 i 位。则

$$\{(x,y):x,y\in S,x\neq y\}=\bigcup_i (U_{i,0}\times U_{i,1})\cup(U_{i,1}\times U_{i,0}).$$

也可以随机划分，每次有至少 $1/2$ 的概率把最值对应的点对 (x^*,y^*) 划分到不同的集合中，此时可以得到正确答案。

做法二. 每个点保留来自两个源的最短路。

做法三. 计算所有 $d(u,S)$ 和 $d(S,u)$ 以及取到最值的最小的 $x\in S$，则最优解路径中存在一条边 $u\to v$ 使得 $d(S,u)$ 的 x 与 $d(v,S)$ 的 y 不同。

证明待补。

评注. *前两个做法可以用于非负权图和负权图。*

4.4.2　重赋权

引理 4.18. 对势函数 $\varphi(u)$，令 $w_\varphi(u,v)=w(u,v)+\varphi(u)-\varphi(v)$。则对于任何 x 到 y 的路径 P，都有 $w_\varphi(P)=w(P)+\varphi(x)-\varphi(y)$。

推论 4.19. $w(P_1)<w(P_2)$ 当且仅当 $w_\varphi(P_1)<w_\varphi(P_2)$；$d_\varphi(u,v)=d(u,v)+\varphi(u)-\varphi(v)$。

引理 4.20. 添加一个向每个点连 0 边的超级源点 s，令 $\varphi = d_s$，则 w_φ 非负。

推论 4.21. 对于有负权的最短路问题，可以运行一次有负权的 SSSP 后将图变为非负权的。

评注. 例如，可以在 $O(n(m+n\log n))$ 时间内计算 *APSP*，这就是 *Johnson* 全源最短路算法。目前的几个比 *Bellman-Ford* 更快的负权图最短路算法都依赖于这个技巧。

4.4.3　最短路树

定义 4.22. 最短路图与最短路树. 在无负环的图中，只保留从 s 出发可达的点和它们之间满足 $d_s(v) = d_s(u) + w(u,v)$ 的边 (u,v) 得到的图称为从 s 出发的**最短路图**。最短路图中以 s 为根的一个树形图称为从 s 出发的一棵**最短路树**。

引理 4.23. 最短路图上任何 $s \to u$ 的路径都是最短路。反过来，任何 $s \to u$ 的最短路都在最短路图上。最短路图上所有的环都是 0 环，最短路图无环当且仅当 s 无法到达任何一个 0 环。

在无非正环的情况下，构建出最短路图后往往可以把问题变为 DAG 上的问题。对于有 0 环的情况，可以先将最短路图进行 SCC 缩点，这样也可以得到一个 DAG。

练习 4.24. 对每对点 u,v，计算有多少条边可以在 u 到 v 的最短路上。$O(n^3)$。

练习 4.25. P3953 [NOIP 2017 提高组] 逛公园. 给定一张无负权图，计算有多少条 1 到 n 的路径的长度不超过 $d(1,n)+K$。

4.5　特殊图上的最短路算法

本节中，我们都假设图无负权。对于边权特殊给定的图，可以用略微修改的 Dijkstra 配合数据结构来求出最短路。

引理 4.26. Dijkstra 取出的 $\tilde{d}(u)$ 不降。

例 4.27. 假设边权只有 K 种，用 K 个队列维护 $\tilde{d}(u)$ 来做到 $O(K(m+n))$。

对于含有大量 0 边的图，下面的推论可以帮助我们忽略这些 0 边。

推论 4.28. 假设 Dijkstra 算法某一步中取出了 $\delta = \tilde{d}(u)$。则 u 只走 0 边能到达的所有 $V\backslash S$ 中的点的最短路都为 $\tilde{d}(u)$。

算法 4.29. 快速处理 0 边的 Dijkstra 算法. 在 Dijkstra 算法中，每次取出 $\tilde{d}(u)$ 后，通过某种方式以 $\tilde{O}$(个数) 的复杂度找到 u 走 0 边能到达的所有 $V\backslash S$ 中的点，将这些点加入 S 并在稍后进行扩展。另外使用一个队列来维护这些加入 S 但还未进行扩展的点。

例 4.30. 给一张图，再给出 q 组特殊连边，每组连边都形如：每个编号在 $[a,b]$ 内的点都向编号在 $[c,d]$ 内的点连边权为 W 的边。

解答. 对每组特殊边都构建两个特殊新点 s_i,t_i，添加以下的边：

- $s_i \xrightarrow{w_i} t_i$
- $\forall u\in[a_i,b_i], u\xrightarrow{0} s_i$，这些边是“隐式”的
- $\forall v\in[c_i,d_i], t_i\xrightarrow{0} v$，这些边是“隐式”的

运行快速处理 0 边的 Dijkstra 算法，则需要支持以下两种操作：

- 对于一个特殊点 t_i，找到 $[c_i,d_i]$ 中所有 $V\backslash S$ 中的点并加入 S。可以使用线段树维护。
- 对于一个普通点 x，找到所有满足 $x\in[a_i,b_i]$ 且 $s_i\notin S$ 的特殊点 s_i 加入 S。可以使用对偶的线段树维护。

对于第一种操作，有以下两种常见维护方式：

- 线段树，每次 $O(\log n)$ 找一个点
- 只需要支持删除和查询前驱，用序列并查集做到线性

第二种操作则更加复杂一些：

- 在线段树上把区间分解后挂在对应的 $O(\log n)$ 个结点上，询问时把从根到叶子挂着的所有区间取出并删除。$O(q\log n)$ 空间。
- 用线段树维护每个结点区间 $[l,r]$ 内满足 $a\in[l,r]$ 的 $[a,b]$ 中最大的 b，每次 $O(\log n)$ 找到一个叶子，然后删除这个叶子上 $b\geq x$ 的区间。每个叶子按 b 排序后就是删除一段后缀。线性空间。

总复杂度 $O((n+m+q)(\log n+\log m))$（优先队列）或 $O(m+(n+q)\log n)$（Fibonacci 堆）。

评注. 读者也许会在其他地方看到所谓“线段树优化建图”的做法，这相当于把我们维护两个操作的过程**显式连边**了。对于无负权的最短路问题，隐式连边的做法在复杂度和常数上总是不劣于显式连边。

4.6　动态最短路

例 4.31. 多次询问：如果在图中添加一条边，新的 $(1,u)$ 最短路是多少？

4.7　常用建模

4.7.1　差分约束

差分约束是指 $x_j-x_i\leq w_{ij}$ 形式的约束。最短路可以用来判断一个差分约束系统是否有解。

定理 4.32. 对每个约束 $x_j-x_i\leq w_{ij}$，以边权 w_{ij} 连边 $i\to j$。差分约束系统有解当且仅当图中无负环。

证明. 若存在负环，则将环上的不等式相加可得 $0 <$ 负数，故无解。反过来，若图中无负环，则添加超级源点 s 并向每个点连 0 边，$x_i = d(s,i)$ 即为一组合法解。 □

推论 4.33. 对任何合法解都有 $x_j - x_i \le d(i,j)$。如果所有 w_{ij} 都是整数，则有解等价于有整数解。

练习 4.34. 通常，我们也允许差分约束系统包含以下非标准形式的约束：

- $x_i - x_j \ge w_{ij}$
- $x_i - x_j = w_{ij}$
- $x_i \le w_i$
- $x_i \ge w_i$

将其改为标准形式。说明如何求出每个 x_i 的最大和最小可行值。

解答. 添加变量 $x_0 = 0$ 即可处理单变量约束，$d_0(i)$ 即为 x_i 的最大可能值，最小可能值则为 $-d(i,0)$。

练习 4.35. P5590 赛车游戏. 给一个有向图，需要给每条边标一个 0 到 9 的整数权值，使得每条从 1 到 n 的路径长度都相同。

4.7.2　分层图

有时候最优化问题具有多个维度，其中某些维度是有自然顺序的，另一些则没有。此时，可以考虑在有顺序的维度上 DP，在没有顺序的维度上运行最短路算法。或者也可以将最短路理解为求解有循环依赖的 DP 的方法，并尽量将依赖性限制在某些维度上。

例 4.36. P4568 [JLOI2011. 飞行路线] 允许把至多 k 条边变为 0，求 s 出发到每个点的最短路。$O(k(m + n\log n))$，要求 $O(n + m)$ 空间。

练习 4.37. k-hop 负权最短路. 在 $O(k(m + n\log n))$ 时间内计算经过不超过 k 条负权边的最短路。

4.8　代数路径

对于普通的最短路，我们计算的是

$$d(s,t) = \min_{P_{st}} \sum_{e \in P_{st}} w(e).$$

不难发现这是 $(\min, +)$ 矩阵乘法下的

$$\bigoplus_{i \ge 0} \mathbf{A}^i.$$

练习 4.38. 计算两两点对之间经过至多/恰好 k 条边的最短路。$O(n^3 \log k)$。

如果将其一般化为任意的 $(\oplus,\odot)$ 半环，就得到

$$f(s,t)=\bigoplus_{P_{st}}\bigodot_{e\in P_{st}}w(e),$$

这就是**代数路径**问题。例如，如果我们取 $(\oplus,\odot)=(+,\times)$，就得到

$$\sum_{P_{st}}\prod_{e\in P_{st}}w(e),$$

这就是所有路径的边权乘积之和。

练习 4.39. 计算两两点对之间经过至多/恰好 k 条边的路径的边权乘积之和。$O(n^3\log k)$。

对于一般的 $(\oplus,\odot)$，无限长度的路径的贡献不一定能够处理。在无负环图最短路这种环路贡献可忽略的半环 $(\oplus,\odot)$ 中，Floyd 与 Bellman-Ford 算法作为动态规划算法可以正确计算答案，而 Dijkstra 算法则需要额外的贪心性质。

练习 4.40. 最大瓶颈路. 令 $(\oplus,\odot)=(\max,\min)$。固定起点 s，以最短路同复杂度计算所有 $f(s,u)$。

练习 4.41. （严格）前 k 短路. 在 k 元素集合/多重集上定义 $A\oplus B$ 为 $A\cup B$ 中的（严格）前 k 小，$A\odot B$ 为 $\{a+b:a\in A,b\in B\}$ 中的（严格）前 k 小，验证这是半环。$(\oplus,\odot)$ 的代数路径对应（严格）前 k 短路。以 k 倍最短路复杂度计算所有 $f(s,u)$。

评注.

例 4.42. 对于一个定义在 S 上的半环 $(\oplus_2,\odot_2)$，令图的边权扩充为 (w,z)，其中 $z\in S$。定义 $(x,a)\oplus(y,b)=\begin{cases}(x,a\oplus_2 b) & x=y\\ (x,a) & x<y\\ (y,b) & x>y\end{cases}$，$(x,a)\odot(y,b)=(x+y,a\odot_2 b)$，验证这是一个半环。构造 $(\oplus_2,\odot_2)$ 使得 $(\oplus,\odot)$ 的代数路径对应

- 最短路计数
- 先按 w_1，再按 w_2 比较的最短路

在正权图上，使用 Dijkstra 算法计算 $(\oplus,\odot)$ 的代数路径。在无负环图上，说明可以在第一维的最短路图上计算第二维的代数路径。

5 特殊的图

5.1 树

5.2 二分图

5.3 DAG

5.4 欧拉图

5.5 竞赛图

5.6 平面图

5.7 广义串并联图

6 网络流

6.1 最大流问题

6.2 费用流

参考文献

ChatGPT 5.6 (Thinking high) 给出了许多结论的简洁证明，与我讨论了许多问题，并对内容进行了审阅。强烈推荐大家使用 Gemini Pro (AI Studio)、ChatGPT Plus 或以上水平的 AI 工具辅助学习。

OI Wiki.

Alex_Wei. 图论 I. 2024.